

TRIANGLE HEALTHY KITCHEN

Software Handover

Prepared by the Software Developer for the Director of Triangle Healthy Kitchen

Customer operations, kitchen production, payments, technical ownership, and outside services

Handover date: 3 October 2026 | Documentation set 1.1

Use this guide to assign owners, verify readiness, and accept the handover with clear evidence.

Contents

Director's Handover Letter and Acceptance Guide	5
A note to the Director	5
What has been handed over	5
Readiness in plain language	5
How the service works from order to delivery	6
Who owns the work	7
Handover actions and evidence	7
The words used in this handover	7
Records, scope, and separate agreements	8
Director's handover record	8
Triangle Healthy Kitchen Software Handover	9
Welcome	9
Read this set in order	9
What this package is and is not	9
How to close the handover	9
Customer Guide	11
Using Triangle Healthy Kitchen	11
Start or renew a plan	11
Payment choices	11
Weekly meals and deadlines	11
Friday meals and delivery	12
Delivery details	12
Language and mobile health data	12
Account help	12
Operations Guide	13
Where each role works	13
Sign-in and role access	13
Role areas in the web portal	13
Daily operating rhythm	13
CEO/Admin	13
Kitchen	14
Transport	14
Rider	14
Escalation rules	14

Kitchen Production Guide	15
The kitchen's work is built around totals first, customer-specific packing second	15
Monthly menu release	15
Customer choice and cutoff	15
Daily production board	15
Weekly bulk purchasing	16
Packing and portioning	16
Portions and recipes to configure before go-live	16
Payments and Subscriptions	17
Source of truth	17
Tap online payments	17
Intended flow	17
If the return page loops or says pending	17
Cash collection	17
Demo records	18
Payment provider and notification setup	18
Platform, Data, and Access Reference	19
System overview	19
Main data areas	19
Access model	20
Data flow	20
Configuration names - values are not included here	20
Web/mobile client build environment	20
Supabase server-side functions and vault	20
Native mobile configuration	20
Integration inventory and state	21
Critical repository cautions	21
Useful source entry points	21
Ownership, Releases, and Maintenance	23
Account ownership	23
Web development and release	23
Mobile build and release	23
Supabase/database/functions release	24
Backups and recovery	24
Routine maintenance calendar	24

Support and maintenance terms	24
Launch Acceptance and Known Limits	25
A. Ownership and access	25
B. Customer signup and account	25
C. Plan, menu, and kitchen	25
D. Payments	26
E. Email, mobile, and integrations	26
F. Release and operations acceptance	26
Known limitations and items requiring acceptance	27
Sign-off	27
Outside Services Setup and Support	28
Service inventory	28
1. Supabase	29
Ownership and initial access	29
Auth and email setup	29
Database, storage, functions, and scheduled jobs	29
If something fails	29
2. Vercel	30
3. Brevo	30
Supabase Auth transactional SMTP	30
THK Edge Function emails	30
4. Tap Payments	30
5. Firebase Cloud Messaging (Android)	31
6. Apple Health and Android Health Connect	31
iPhone / Apple Health	31
Android / Health Connect	31
7. Location and maps	32
8. WhatsApp, social links, and remote images	32
Provider incident checklist	32

Director's Handover Letter and Acceptance Guide

To: Director, Triangle Healthy Kitchen **From:** Software Developer **Handover date:** 3 October 2026 **Product:** Triangle Healthy Kitchen website, customer mobile apps, and operations system

A note to the Director

I have prepared this handover so you and your team can understand what the software is intended to do, how the daily work moves through it, which outside accounts support it, and what must be checked before you rely on it for live customers. It is written for Triangle Healthy Kitchen first. The technical instructions later in the book are included so that you can direct an employee or future developer without losing control of your accounts or system.

The package includes the customer guide, operating guides for management and each work team, the kitchen production guide, payment and subscription guidance, a technical reference, a release and maintenance guide, a launch checklist, and a runbook for outside services such as Supabase, Vercel, Brevo, Tap, and Firebase. The PDF is the convenient reading copy. The Markdown files are the editable text source.

What has been handed over

Delivered area	What your team can use it for
Public website	Introduce Triangle Healthy Kitchen, explain plans, and provide the customer account and subscription experience. The live URL recorded for this handover is <code>https://trianglehealthykitchen.vercel.app/</code> .
Customer account and mobile app	Let customers register or sign in, provide their profile, select meals, enter a delivery address, see account information, and use the available wellness features. The Android and iOS app projects are in the repository; source code is not inside this document ZIP.
Operations website	Give CEO/Admin, Kitchen, and Transport their role-specific web workspaces. Business operations dashboards belong on the web. Riders use the mobile app.
Menu and kitchen workflow	Maintain the dish catalogue and weekly/monthly menu, receive customer choices, produce day-level meal and portion totals, calculate ingredient estimates from recipes, show kitchen safety notes, and support packing. The kitchen must approve recipe amounts and portion ranges before procurement relies on the totals.
Payments	Provide Tap checkout and a staff-controlled manual review for a captured Tap payment, plus a pending cash request that only activates after a staff member confirms cash was received. The live Tap account, webhook, and end-to-end production transaction still need client-side verification.
Outside-service instructions	Explain how to maintain the connected provider accounts without putting credentials into this package.

Readiness in plain language

A feature appearing in the source code or on a screen does not prove that it is configured, deployed, tested, or ready for production. The website is live, but the following items remain conditions to verify before you treat the complete service as production-ready:

Item	What is known at handover	What must happen before relying on it
Online payments	Tap checkout, callback, and a staff reconciliation control are present in the reviewed source.	Confirm the client-owned Tap account, live/test mode, callback and webhook, and a real authorized transaction from checkout through active plan. Never ask a customer to pay twice while a charge is being checked.
Cash payments	Cash requests can wait for review; the plan should stay inactive until cash is actually received and an authorized Admin/CEO confirms it.	Train the staff who collect and confirm cash. Match the system entry to the cash received and to the finance records.

Item	What is known at handover	What must happen before relying on it
Menus and reminders	Menu publication and scheduled menu-choice/reminder code are present.	Confirm the correct menu appears to customers on Saturday, Kitchen's choice fills missed selections after Thursday's cutoff, and the reminder is sent about 24 hours before cutoff. These depend on production database jobs and email setup.
Kitchen purchasing	The system can total ingredients from recipe quantities and selected meals.	Kitchen must replace/approve seeded estimates, check units and substitution behavior, and enter the agreed gram ranges for portions A-F. Until signed off, totals are planning aids, not approved purchase orders.
Staff access	CEO/Admin, Kitchen, and Transport web workspaces are intended; Rider and Customer use the mobile app.	Create named staff accounts, check each role on the live system, and confirm data permissions. The reviewed web source did not show a clear control for approving pending rider applications; confirm the actual rider approval process before onboarding riders.
Email and external services	Brevo, Supabase, Vercel, Tap, Firebase, and health/location integrations have source-level setup notes.	The Director or named account owners must verify account access, secrets, sender/domain configuration, scheduled jobs, and real-device behavior.
App stores and mobile health	Android and iOS projects exist.	Store publication requires the client's own developer accounts, signing, declarations, review, and device testing. iOS HealthKit needs Apple's paid developer membership and signing. This handover does not claim either app is approved or published in an app store.
Arabic	Arabic language support is present.	Complete a screen-by-screen review of customer and role-specific flows in Arabic before advertising the translation as complete.

The one-day and six-day trial offers are described as three meals and one snack. The additional offer discussed with Triangle Healthy Kitchen is 1,600 kcal, three meals and two different snack choices, at QAR 2,499. The monthly Friday add-on discussed is QAR 199 and includes Friday meal selections and Friday delivery for the subscription month. Treat these as business-approved prices and rules only after confirming the active package and checkout records display them correctly. The active package records control what customers are actually offered. The monthly plan duration is 24 service days, according to the current product rules. Sadad was not found as an integrated checkout in the reviewed source.

How the service works from order to delivery

- 1. The Director or Admin keeps the offer current.** Active package records define what is for sale, the price, duration, included meal periods, and any Friday eligibility.
- 2. The customer subscribes and chooses meals.** The customer completes the profile, keeps or changes the preselected Kitchen's choice, adds allergy and preparation notes, enters the place where food should arrive, then chooses an offered payment method. A customer should not need to change a meal choice they already accept.
- 3. A payment is checked before activation.** A Tap plan becomes active when the verified payment status is captured. Cash stays pending until cash is physically received and an authorized Admin/CEO confirms it. Pending does not mean paid or ready for service.
- 4. Menu choices become kitchen production totals.** Kitchen needs day-specific dish and meal counts, bulk ingredient estimates, portion counts, and the allergy/avoid/preparation information needed to cook safely. Kitchen does not need customer addresses or a full customer directory to purchase ingredients.
- 5. Kitchen portions and Packing uses the detail.** Once bulk food arrives, staff use packing details to match dishes, requested modifications, allergy notes, and the configured A-F portion category to the correct package.
- 6. Transport plans one delivery stop per customer for that service day.** All that day's meals travel together. A zone is inferred from the customer's chosen map location when the lookup works; staff must resolve missing or uncertain zones before assigning a route.
- 7. The customer receives the daily delivery and manages the account.** Staff should keep the operational status current and resolve payment, menu, address, or delivery exceptions in their designated workspace.

Who owns the work

Owner	Responsibility
Director / business owner	Owns business decisions, provider accounts, approved prices and terms, acceptance, support arrangements, data/privacy decisions, and who can access production.
Admin	Maintains packages, reviews customers and payments, checks operational alerts, manages staff access as permitted, and coordinates exceptions.
Kitchen lead	Approves dish catalogue, recipes, quantities, units, A-F ranges, menus, daily production and packing procedures, and allergy/preparation handling.
Transport lead	Confirms address/zone information, groups stops, assigns riders, and follows up on delivery exceptions.
Rider	Uses the assigned mobile workflow to complete stops and report accurate delivery status.
Named technical owner	Maintains deployments, Supabase database/access rules, outside-service settings, mobile builds, backups, and incident response. This can be the developer or another provider appointed by the Director.
Finance owner	Reconciles Tap settlements, cash receipts, refunds/disputes, receipts, and accounting records.

One person may hold more than one business role, but the system should still use individual named accounts. Do not give a role more customer, health, payment, or address information than its work requires.

Handover actions and evidence

Use the checklist in **07 - Launch Checklist and Known Limits** as the acceptance record. For each item, the assigned owner records Pass, Fail, or Not tested, the date, and a safe evidence reference. A passing visual check alone is not sufficient for payment, role security, scheduled jobs, backups, or store builds.

The first acceptance should confirm: the Director can access and recover the provider accounts; each role can sign into the right workspace; the customer flow respects the package limits; menu publishing and the Thursday/Saturday schedule work in the live database; Kitchen can see day totals and relevant safety notes; Transport can resolve zones and assign a single stop per customer/day; a payment is reconciled exactly once; and Arabic and mobile layouts have been reviewed on the intended devices.

Do not use live customer data as test data. Demo customers are synthetic, unpaid records and must remain excluded from financial revenue. Do not delete live data for a demonstration. Use a test account or a clearly marked demo dataset, then clean up only the records identified as demo.

The words used in this handover

- **Service day / service week:** The day or Saturday-starting week for which meals are prepared and delivered. This is separate from the calendar date on which a monthly package is purchased.
- **Meal period / meal slot:** A meal category such as breakfast, lunch, dinner, or snack on a given service day. Package limits decide which periods the customer can select.
- **Kitchen's choice:** The kitchen-selected default meal for a slot. It is preselected for the customer where available and can fill a missed choice after the weekly deadline if the production schedule is configured.
- **Zone:** The delivery area used to group customer stops and assign riders. A map lookup can suggest a zone; a suggestion still needs review when blank or uncertain.
- **Portion class A-F:** The kitchen's agreed portion band for an individual packed meal. The business must provide the gram range for each letter. The range is not a medically prescribed serving size.
- **Recipe quantity:** The ingredient amount and unit assigned to one serving of a dish. Ingredient totals are calculated from these values, so inaccurate recipes produce inaccurate purchasing estimates.
- **Production total:** The daily aggregate of dishes/meals/portion counts needed by Kitchen. It is deliberately different from the customer-by-customer Packing list.
- **Supabase:** The platform's sign-in, database, file storage, server-side functions, access rules, and scheduled jobs.

- **Vercel:** The hosting/deployment service for the public website and web operations portal.
- **Edge Function:** Server-side code hosted by Supabase. It keeps payment and email secrets off the customer app.
- **RLS (row-level security):** Database rules that decide which records each signed-in role may read or change. Hiding a screen tab is not a replacement for these rules.

Records, scope, and separate agreements

This handover explains the software and operational setup; it is not a price list, a payment guarantee, a support SLA, or legal advice. The fee, license, referral, ownership, support, maintenance, and app-store responsibilities are governed by the signed commercial agreements. The supplied agreements were described as drafts with blanks; complete and reconcile them before signing. This manual does not fill in missing contract terms.

This documentation bundle does not contain source code, customer exports, provider passwords or API keys, Firebase configuration, mobile signing keys, recovery codes, or signed commercial agreements. Those items belong in the client-owned repository/provider accounts and approved secure storage.

Director's handover record

Record	Complete together
Director / authorized owner	Name: _____ Date: _____
Business operating lead	Name: _____ Date: _____
Named technical owner	Name: _____ Date: _____
Finance/payment owner	Name: _____ Date: _____
Kitchen lead sign-off	Name: _____ Date: _____
Transport lead sign-off	Name: _____ Date: _____
Items accepted	Checklist/reference: _____
Items still open	Checklist/reference and owner: _____
Support/maintenance agreement	Document/reference: _____

Triangle Healthy Kitchen Software Handover

Prepared by: Software Developer **Prepared for:** Director, Triangle Healthy Kitchen **Handover date:** 3 October 2026
Documentation set: 1.1

Welcome

Director, this is the practical handover for the Triangle Healthy Kitchen website, customer apps, and operations system. I have arranged it around your team's work: what customers do, what each work area receives, how the menu and kitchen production totals connect, how payments are checked, and which account owners must complete the final setup.

Start with **00 - Director's Handover and Acceptance**. It explains the service from customer signup through kitchen and delivery, identifies what this software package includes, and lists the items that must be verified before live operation. The remaining guides provide the detail for your customer support, Admin, Kitchen, Transport, finance, and technical owners.

Read this set in order

1. **00 - Director's Handover and Acceptance** - the full delivery summary, owner responsibilities, definitions, acceptance evidence, and readiness conditions.
2. **01 - Customer Guide** - how customers register, choose packages and meals, set an address, pay, use health connections, and get account help.
3. **02 - Operations Guide** - what the CEO/Admin, Kitchen, Transport, and Rider teams do and which workspace they use.
4. **03 - Kitchen Production Guide** - menu release, daily cook totals, purchasing estimates, allergy and preparation notes, portions, and packing.
5. **04 - Payments and Subscriptions** - package rules, Tap reconciliation, cash collection, and safe handling of pending plans.
6. **05 - Technical Handover** - where the platform's main parts live, what data they handle, and which access rules protect it.
7. **06 - Deployment and Maintenance** - how Triangle Healthy Kitchen releases, backed up, maintained, and supported.
8. **07 - Launch Checklist and Known Limits** - the evidence-based acceptance record for final business sign-off.
9. **08 - External Services Runbook** - how to manage Supabase, Vercel, Brevo, Tap, Firebase, mobile health, maps, WhatsApp, and remote media.

What this package is and is not

This is the operating and technical handover for the software and setup status described in this handover. It is not a claim that every external account is configured or every feature has passed a live production test. The website is live at trianglehealthykitchen.vercel.app. The iOS and Android projects exist in the code repository, but source code is not bundled here and app-store publication is not established by this manual.

The PDF is designed to read and share. The Markdown files are editable source documents. Neither copy contains private credentials, signing keys, customer exports, or signed agreements. Keep those in client-controlled accounts or approved secure storage.

How to close the handover

The Director assigns a named business owner to each item in `07-Launch-Checklist-and-Known-Limits.md`. The owner records Pass, Fail, or Not tested, with a date and safe evidence. The Director accepts the work only after all launch-critical items have passed or an explicit written exception names its owner and consequence. Payment configuration, data access, menu schedules, recipe/portion accuracy, backups, and app-store readiness must not be inferred from a successful screen rendering.

Commercial terms remain in the separate fee, license, referral, and services documents. Complete and align their blanks before signing; this manual does not supply missing prices, warranties, ownership rights, support hours, or service levels.

Customer Guide

Using Triangle Healthy Kitchen

Customers can use the live website or the iOS/Android app to review plans, create an account, choose meals, provide delivery details, and manage an active subscription. The mobile app is for customer and rider tasks; company operations dashboards are on the web.

Start or renew a plan

1. Open the website or app and choose **Start your plan**.
2. Complete the profile and health-goal questions. Enter realistic age, weight, and height values. A BMI report is optional; it is shared for the nutrition team's review and is not a substitute for medical advice.
3. Choose a package. Package availability, name, price, meal periods, and duration are loaded from the active package records. Confirm the current values shown in the app before paying; the code's fallback examples may not match later admin-edited package records.
4. Choose the available meals for the upcoming service week. The kitchen's choice is preselected where one is available. Keep that selection if you are happy with it; change only the meals you want to change.
5. Provide allergies and kitchen preparation notes. These are shared with Kitchen for food safety and preparation. Describe ingredients to avoid clearly; contact the business directly for urgent allergy questions.
6. Enter the place that should receive the food: home, office, gym, or another location. Use **Locate** if you are physically there, or enter the address manually if you are elsewhere. Review the building, street, area, and map pin. Confirm the suggested delivery zone with the business if the address is not recognized.
7. Create/sign in to your account, accept the displayed terms, and choose the payment method offered.
8. Keep the confirmation/reference. If checkout returns without showing an active plan, sign in and check the plan status before trying to pay again. A pending payment should be reconciled by Admin/CEO first to avoid duplicate charges.

Payment choices

- **Tap:** Continue to Tap's secure checkout. The plan activates after a verified capture. If the return page says pending, do not assume the payment failed; contact the business with the transaction reference so staff can check Tap and the operations payment log.
- **Cash collection:** The request remains pending until Admin/CEO confirms that cash was physically received. The confirmation message says the team will reach out. Do not treat an unverified cash request as an active plan.
- Sadad is not documented as an available integrated checkout in the current source. Use only the payment options actually displayed.

Weekly meals and deadlines

- The service week starts Saturday. The menu is intended to open on Saturday for the upcoming week.
- Change meal choices before **Thursday 11:59 PM Qatar time**. The app should preselect Kitchen's choice when available, so customers who accept it do not need to reselect it.
- After the deadline, the scheduled kitchen-choice process fills any missing meal choices. Confirm that the client's production environment has the scheduled database job enabled; see the Launch Checklist.
- A reminder email is intended to go out about 24 hours before the deadline to customers with incomplete meal choices. Email delivery depends on the production Brevo configuration and scheduled job.
- If there is no new release for the upcoming week, the system has a database function to roll the latest prior menu forward. Operations should still check the menu before the week opens and confirm the fallback job is active.

Friday meals and delivery

The optional Friday add-on is **QAR 199 per monthly subscription** and includes Friday meal selections and the Friday delivery. It is not an extra charge per Friday. It is not offered with the one-day or six-day trial in the current flow. Confirm the total displayed before payment.

Delivery details

Each customer receives **one delivery stop per service day**, with all meals for that day together. The delivery address and zone help Transport group stops and assign riders. If you move between home, office, and gym, update the active delivery address/details through the account flow as supported and tell the business about a change that affects an imminent route.

Language and mobile health data

- Use the language control to switch between English and Arabic. If wording appears untranslated or confusing, capture the screen and report the page and language; do not assume a partial translation changes plan or payment rules.
- Health sync is optional and customer-initiated. iPhone uses Apple Health/HealthKit; Android uses Health Connect. The customer must grant read access on their device. Samsung Health data must be shared with Health Connect first. If unavailable, continue using the product without syncing. The app requests steps, distance, calories, and weight permissions in the mobile implementation; actual data availability depends on device, OS, permissions, and signing/configuration.

Account help

Use the password-reset link sent to the email on the account and open it on the live domain. A reset page that reports a missing/expired auth session needs a fresh recovery email and valid session; do not repeat the password update from an old link. For a plan that is pending after payment, ask Admin/CEO to reconcile the transaction before starting another checkout.

The account has a **Request account deletion** flow. The app tells the customer the request is processed within 30 days. Separately, the database contains a scheduled cleanup for non-owner, non-staff, non-rider customer accounts whose subscription period ended more than 30 days earlier. Confirm production scheduling and the published privacy policy before promising retention/deletion dates beyond what the screen says.

Operations Guide

Where each role works

Role	Main workspace	Intended work
CEO	Web operations portal	Oversight and all operating areas.
Admin	Web operations portal	Customers, subscriptions, menus, payments, reports, team/settings, and daily administration.
Kitchen	Web operations portal	Menu catalogue/release, daily production, ingredient purchasing totals, allergies/preparation notes, and packing.
Transport	Web operations portal	Delivery addresses/zones, rider assignment, routes, delivery status, and rider fleet.
Rider	Mobile app	Assigned stops, delivery execution, and availability.
Customer	Website or mobile app	Subscription, meals, delivery details, health/rewards, and account settings.

The mobile app routes CEO/Admin/Kitchen/Transport to the web portal. Do not look for a Kitchen or Transport dashboard inside the mobile app. Riders are the operations role intended to work in mobile.

Sign-in and role access

1. Use the staff member's own approved sign-in account. Do not share a personal owner account among staff.
2. The web role is read from the authenticated staff-role record; the interface is not the only security boundary. Supabase policies/views/functions must also be present and current.
3. After sign-in, confirm the workspace label and tabs match the staff member's role. If access is wrong, stop and ask the system administrator to correct the role assignment; do not work around it by sharing another role's credentials.
4. Sign out on shared devices. Remove access promptly when a staff member leaves.

Role areas in the web portal

- **CEO/Admin:** Dashboard, Customers, Subscriptions, Meal Plans, Kitchen, Packing, Delivery, Drivers, Payments, Reminders, Reports, Consultations, Team, Settings. Actual visible tabs are role-filtered.
- **Kitchen:** Dashboard, Meal Plans, Kitchen, Packing, and Reports are the work areas relevant to food preparation. If the Kitchen account shows Consultations, remove it from Kitchen navigation and verify that the role cannot access consultation records. Consultations belong with CEO/Admin.
- **Transport:** Dashboard, Delivery, Drivers.

The web fleet view currently reads approved rider records. A pending-application approval control was not identified in the operations workflow reviewed for this handover. Before inviting riders, the Director must assign an owner and verify the actual approval route in the production system. Do not assume that submitting an application makes a rider approved.

Daily operating rhythm

CEO/Admin

- Review alerts for pending payments, consultation requests, incomplete customer zones, pause/resume requests, and failed email events.
- Check new subscriptions and payment status before telling a customer the plan is active.
- For Tap manual verification, inspect the exact transaction in Tap first. Confirm it is captured, the reference matches, and amount/currency match. Only then use **Verify captured in Tap**. This action activates the plan and is audited.
- For cash, confirm cash was physically collected before using **Confirm cash received**. This activates the plan and is audited.

- Review package price/duration and current menu before advertising signup.
- Activate demo customers only for presentation/operations previews. Demo records remain marked synthetic and unpaid, and never count as cash or Tap revenue.
- Keep Customer exports and consultation/health information restricted to staff who need it.

Kitchen

- In **Meal Plans**, manage the dish catalogue, ingredient recipe configuration, monthly menu, review, and publication.
- In **Kitchen**, pick the service week and day. Use **Today's production** to focus on the selected day instead of scanning every day's dishes.
- Use dish/meal/portion counts for preparation. Read the parallel allergy, avoid, and preparation notes before cooking.
- Use **Weekly bulk purchasing** to see aggregated ingredient totals. Seeded recipe quantities are estimates until verified by the kitchen; do not place a purchase order from unvalidated estimates.
- Use **Packing** for the client-specific portion/dish detail after bulk ingredients arrive. A-F portion ranges are configurable in Settings and need Triangle Healthy Kitchen's approved gram ranges entered.

Transport

- Choose the delivery date and review zone/address grouping. Resolve missing or ambiguous zones before route assignment.
- A customer gets one stop with all meals for that day. Friday stops appear only for monthly customers with the Friday add-on.
- Assign one rider to each stop; use the map link to confirm the location. Record delivered/failed status promptly.
- Do not use kitchen screens to resolve customer zones; those belong to Transport/Admin.

Rider

- Sign in through the mobile app after the application is approved.
- Set availability/online state as supported, review assigned stops, use the address/map details, and update delivery status honestly.
- Contact Transport/Admin for an unsafe, inaccessible, or incorrect address. Do not mark a stop delivered until it has been delivered.

Escalation rules

- **Payment says pending after customer paid:** capture transaction reference and amount; check Tap dashboard and payment event log; use manual review only after verifying captured status. Do not tell the customer to purchase again first.
- **No menu appears:** check Meal Plans publication and available-from date; confirm active collection, archived fallback, and production database migrations.
- **Kitchen lists empty:** confirm active paid/demo records, meal selections for the Saturday service week, and valid dish IDs; confirm demo customers were activated for display where necessary. Check recipe config separately for ingredient totals.
- **Rider missing:** confirm rider approval process and active rider record. Current code does not prove that pending applications can be approved in the dashboard.
- **Data/API error:** save the time, role, page, and request/error code; do not attach keys or personal customer data. Escalate to the technical owner.

Kitchen Production Guide

The kitchen's work is built around totals first, customer-specific packing second

Kitchen needs to know what to purchase and prepare for each service day without opening individual customers' records one by one. The operational views separate bulk totals from the details needed to portion and prepare safely.

Monthly menu release

1. Keep the dish catalogue accurate: names, Arabic names where used, active status, calories, ingredients, and recipe quantity/unit per serving.
2. Open **Meal Plans** and start a monthly menu from the catalogue (or import the current PDF if that workflow is available in the deployed build).
3. Review extracted/imported dishes against the source menu. Correct names, meal periods, day, and availability; add or remove dishes as needed.
4. Mark the Kitchen's choice among the available options for each day/meal slot. Customers who miss the weekly deadline use this choice.
5. Review the monthly schedule and publish. The release is expected to be available to customers on Saturday for the service week ahead.
6. Sign in with a customer test account and confirm the published week is visible before relying on it. If no replacement release exists, the backend contains a function to roll a previous release forward, but that job and active collection must be verified on production.

The user interface describes a monthly menu repeating across four service weeks. Confirm the number and dates in the deployed schedule when publishing; monthly date boundaries and Saturday service-week boundaries are not identical.

Customer choice and cutoff

- Customers make meal selections for their package before payment and can manage the next service week in their dashboard.
- Menu options are attached to a service day and meal period. A package limits which meal periods are available; Friday is added only for the eligible monthly Friday add-on.
- Existing Kitchen's-choice defaults are preselected. Customers should not have to reselect unchanged defaults.
- The selection window closes Thursday at 11:59 PM Qatar time. A database job is intended to apply defaults just after the Qatar cutoff. Verify the job in Supabase rather than relying only on the UI message.

Daily production board

Select the service week and the day being cooked. The **Today's production** section should show the selected day's required dishes grouped by dish/meal and portions. Demo records are visually marked **DEMO · UNPAID**; distinguish them from real paid customers in any production handoff.

The safety/preparation section summarizes the details Kitchen must not miss:

- allergies;
- ingredients to avoid or dislikes;
- meal-specific modification or preparation notes;
- selected dish and meal period;
- A-F portion category and count, when configured.

Use these notes to safely plan and portion orders. Allergy flags are not a guarantee that cross-contamination is prevented; the kitchen needs its own food-safety process.

Weekly bulk purchasing

The ingredient-order view aggregates configured recipe quantities over selected active meal records. Conceptually:

$\text{purchase quantity} = \text{sum}(\text{recipe quantity per serving for each selected serving, after removals/substitutions})$

The output is an ingredient total and the number of meals that use it, not a customer-by-customer purchase list. It can change when menu selections or recipe quantities change.

Important: recipe quantities seeded in the system are estimates. Kitchen must verify each recipe quantity, unit, edible yield/waste assumptions, and portion scaling before treating the computed amount as an authoritative purchase order. A zero/empty total may mean a dish has no recipe configuration, no current selections, inactive/non-matching dishes, or data has not loaded; it does not necessarily mean the kitchen needs zero ingredients.

Ingredient totals in source are based on configured per-serving ingredient quantities and customization logic. Confirm that portion classes A-F correctly affect actual ingredient ordering in the deployed recipe model before relying on different gram sizes for procurement.

Packing and portioning

After bulk items arrive:

1. Open **Packing** for the same service week/day.
2. Use the customer/dish detail to assemble the requested meals.
3. Follow the recorded preparation notes, allergies, avoided ingredients, substitutions, and A-F category.
4. Use the saved A-F min/max gram ranges as guides only after the kitchen confirms them in Settings. Empty ranges are intentionally shown as unset.
5. Reconcile packed meal counts against the production totals before closing the sheet.

Portions and recipes to configure before go-live

- Enter client-approved gram ranges for A, B, C, D, E, and F.
- Review every active dish ingredient list and per-serving quantity; replace seeded estimates with kitchen-approved values.
- Verify units are consistent (g, kg, ml, pieces, etc.) and substitution/removal recipes produce the expected totals.
- Run a controlled sample with two or more customers choosing the same dish at different portion levels and with different notes. Confirm the bulk purchasing total, daily production grouping, safety notes, and individual packing breakdown all make operational sense.
- Sign off the printed/exported production and packing views with the kitchen lead.

Payments and Subscriptions

Source of truth

The production packages records are the active source for package name, price, currency, meal periods, duration, and availability. The web/mobile code includes fallback package examples for resilience; those examples are not proof that current database values match them. Admin should verify packages in the web Subscriptions area before publishing prices.

Known business rules reflected in source/product decisions:

- Monthly subscriptions use 24 service days.
- The one-day and six-day trial offers provide three meals and one snack, subject to current package records.
- The additional 1,600 kcal offer is described as three meals plus two different snack choices at QAR 2,499; verify the active database record before quoting.
- Monthly Friday add-on is QAR 199 for Friday meals/selections and delivery for the month, not per Friday. Trials are excluded.

Do not use this document as a price list when the active database record or customer checkout shows a different current price.

Tap online payments

Intended flow

1. The customer chooses a package, menu, address, and Tap as payment method.
2. A server-side Supabase Edge Function creates the Tap charge using server secrets.
3. Tap returns the customer to `/payment/callback?tap_id=...`; a webhook/callback updates the transaction and plan state.
4. The customer app waits for captured status. A captured payment activates the plan; a pending result stays pending.

If the return page loops or says pending

1. Do not make the customer purchase again immediately.
2. Record the customer account email and exact Tap reference/charge ID, amount, currency, and time. Do not copy card data.
3. Check the exact charge in the Tap dashboard and the web **Payments** event log.
4. If the Tap dashboard confirms that exact charge is captured and amount/currency match the transaction, a CEO/Admin may use **Verify captured in Tap**. That is a privileged manual reconciliation action, not a substitute for looking up the charge.
5. If not captured, failed, or mismatched, leave the plan unactivated and resolve through the payment provider.
6. Ask the customer to sign in again and refresh plan status after the review. The auth layer rechecks server-side subscription access when the page regains focus.

Tap checkout and manual reconciliation code exist. This handover review does not certify live merchant credentials, Tap account configuration, deployed webhook delivery, refund/dispute handling, or a successful production end-to-end payment. Perform a low-value authorized live transaction and reconcile/refund it under an approved process before launch.

Cash collection

1. Customer chooses cash and submits the subscription request.
2. The payment transaction is pending; the plan remains inactive.
3. Staff contacts the customer and collects the full amount before the plan begins.

4. Only after receiving the cash, a CEO/Admin opens **Payments** and clicks **Confirm cash received** for that transaction.
5. The database function checks payment method, status, package amount/currency, and authorized staff role; it activates the plan and writes an audit event.
6. Reconcile physical cash against the system report according to Triangle Healthy Kitchen's bookkeeping process.

Never verify cash because a customer says it was handed over; verify only after the responsible employee confirms actual collection. Do not let Kitchen or Transport approve a cash transaction.

Demo records

Demo customers are synthetic and unpaid. Admin/CEO can activate them for kitchen/routing previews where supported. Activation must not be treated as payment, cash collection, or real revenue. Keep demo records clearly marked in operations and exclude them from financial reports.

Payment provider and notification setup

- Tap is the only online checkout integration evidenced by the reviewed current product source. No Sadad implementation was found.
- Tap account keys and merchant settings are stored server-side through Supabase-managed secrets/vault access; never enter them into client code or this documentation.
- Brevo-backed functions send operational email/reminders; the sender identity, API secret, domain verification, suppression handling, and production cron invocation must be checked.
- Payment settlement, refunds, chargebacks, tax/accounting, and cash reconciliation procedures remain Triangle Healthy Kitchen's financial responsibility unless a signed services agreement says otherwise.

> **For the Director and the named technical owner.** This section shows where the parts of the system live, how customer and operations data move, and what a future maintainer must check before changing it. You do not need to edit these files for ordinary daily operations. Use this map when appointing or briefing the technical owner; keep account ownership with Triangle Healthy Kitchen.

Platform, Data, and Access Reference

System overview

Component	Repository location	Technology / responsibility
Customer and operations website	thk-web/	React, TypeScript, Vite, Supabase JS, Vercel. Public site, customer account, subscription, menu, and role-filtered operations portal.
Mobile application	thk-mobile/	React, TypeScript, Vite, Capacitor 7. Customer and rider app; operations staff are sent to the web portal.
Database and access rules	thk-web/supabase/migrations/ and duplicated migration tree under thk-mobile/	PostgreSQL schema, RLS, views, triggers, RPCs, scheduled jobs. Review migration history before applying changes.
Server-side functions	thk-web/supabase/functions/ and thk-mobile/supabase/functions/	Supabase Edge Functions for payment, webhook, reminders, booking email, health activity reconciliation, and account deletion. The copies are not complete mirrors.
Older prototype	Repository root app.js, index.html, styles.css, JSON translations	Earlier static prototype. It is not the current production app; make current product changes under thk-web/ and thk-mobile/.

Production web URL: <https://trianglehealthykitchen.vercel.app/> Supabase project reference used in source: `teguqlkfmchxucedxvpu` Mobile app ID: `com.trianglehealthykitchen.app`

Main data areas

Names below are the principal data concepts visible in the application and database migration files reviewed for this handover; this is an operational map, not a complete column-level schema export.

Data area	Purpose and primary users
auth.users, subscribers, staff_members	Identity, customer subscription profile, account state, and staff roles. Auth user creation alone does not mean a paid customer plan is active.
packages	Current package price, currency, duration, meals, availability, and display properties. Active database rows take precedence over code fallback samples.
dishes, ingredients, meal_ingredient_config	Shared menu catalogue, ingredient catalogue, and recipe quantities/units used in purchase totals.
monthly_menu_documents, menu_availability, weekly_menu_selections	Uploaded/reviewed menu source, scheduled dish availability, and each service-week meal choice.
kitchen_production_view, kitchen_ingredient_order_view	Aggregated production/safety and bulk ingredient views for Kitchen; restricted to authorized operations roles.
payment_transactions, payment_logs	Payment request/callback/reconciliation status and audit events. Do not edit production rows manually to bypass guarded RPC workflows.
rider_applications, rider_deliveries	Rider identity/approval and assigned delivery stops/status. The web source reads approved riders for fleet views; pending approval workflow needs confirmation.
notifications, weekly_menu_reminders	Email outcomes and per-customer/service-week reminder tracking.
subscription_pause_requests, consultation/provider booking records	Customer requests and consultation review queues for CEO/Admin.
Health/activity records and BMI report storage	Customer-provided wellness data and optional uploaded health report; treat as sensitive personal data.

Access model

- Staff role values in the app are CEO, Admin, Kitchen, and Transport. Rider is a separate application/approval role. Subscriber is the customer role.
- The web app filters tabs by role and obtains staff role through the `current_staff_role` database RPC.
- Supabase RLS, staff checks, guarded RPC functions, and restricted views are intended to enforce access at the data layer too. Never treat a hidden tab as authorization.
- `transport_subscribers_view` is a narrow transport-specific view; `kitchen_subscribers_view` intentionally exposes only kitchen-relevant active/demo status metadata. Kitchen production and safety views supply meal, portion, allergy, avoid, and prep information without making the full customer directory a kitchen tool.
- Manual payment RPCs are CEO/Admin-only. Review policies/RPC definitions whenever staff roles change.
- Exports may include personal data. Limit export rights and retention to named staff with an operational need.

Data flow

1. Customer signs in and provides subscription, profile, menu, delivery, and payment details.
2. Package, menu, identity, and payment records are validated by the app and server/database constraints.
3. Tap callback/webhook or authorized cash-review RPC updates transaction and plan state.
4. Weekly customer meal choices feed Kitchen's day-specific production totals, ingredient order totals, allergy/preparation notes, and packing details.
5. Delivery address/zone feeds Transport's route list; one rider assignment is created per customer stop/day, not once per meal.
6. Reminder/booking functions write email delivery outcomes for Operations to review.

Configuration names - values are not included here

Web/mobile client build environment

- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY` (public client key; never use the service-role key here)
- `VITE_OPERATIONS_WEB_URL` (optional mobile override; default is the production Vercel domain)

The web has a source fallback for the public Supabase URL/key. Keep deployment values aligned with the intended project. Do not paste any actual keys into handover documents.

Supabase server-side functions and vault

The reviewed functions refer to configuration such as `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `PUBLIC_APP_URL`, Tap configuration (`TAP_SECRET_KEY`, merchant config through protected database helpers), and Brevo configuration (`BREVO_API_KEY`, `SENDER_EMAIL`, provider notification destination). Exact storage path differs between functions: some use Edge Function secrets and some call protected database/Vault helper RPCs. Inspect each deployed function before rotating credentials. `SUPABASE_SERVICE_ROLE_KEY` and Tap/Brevo secrets must remain server-side.

Native mobile configuration

- Android Firebase uses `thk-mobile/android/app/google-services.json`. The local file is excluded from this documentation bundle and should be supplied from the client-owned Firebase project through a secure channel. Its presence enables Android push setup in the Vite build configuration; absence intentionally skips push registration.
- iOS health sync needs the HealthKit capability, usage description, correct Apple App ID, provisioning profile, and signed build. Store account access is controlled by the client.
- Android health sync uses Health Connect; users with Samsung Health must enable its Health Connect sharing.
- Device location/notifications/health permission prompts must be tested after producing signed builds.

See **External Services Runbook** for step-by-step ownership, setup, safe credential handling, testing, and failure procedures.

Integration inventory and state

Integration	Source evidence	Configuration/acceptance still required
Supabase Auth/Database/Storage	Used by both apps and Edge Functions.	Verify deployed schema/policies, storage access for BMI reports, project ownership, backup plan, and auth email/phone settings.
Vercel	Vite production build and SPA rewrite configuration exist.	Confirm domain, project ownership, environment variables, analytics setting, and production deployment commit. Browser extensions can block Analytics without affecting app behavior.
Tap	Checkout function, callback page, webhook implementation, and manual review exist in duplicated function trees.	Reconcile canonical source, deploy webhook, validate Tap dashboard settings/secret, and complete authorized captured-payment test plus callback/manual-reconciliation cases.
Sadad	No current checkout code located.	Not available until implementation, credentials, callback, security review, and end-to-end verification are separately completed.
Brevo	Booking notification and menu-reminder function code exists in mobile function tree and migration scheduling refers to send-reminders.	Confirm canonical deployed code, sender/domain, vault settings, delivery logs, recipient behavior, and scheduled invocation.
Firebase Cloud Messaging	Android push registration is conditional on local Firebase config.	Configure client-owned Firebase app, Android package, file, permissions, and real-device push test.
Apple Health / Health Connect	Capacitor health plugin integration in mobile source.	Complete platform capability/signing and real-device permission/read/sync tests.
Maps/geolocation	Google Maps URL/coordinate parsing and links, OpenStreetMap Nominatim reverse geocoding, and Qatar GIS FeatureServer zone lookup exist in source.	Test Qatar address lookup, manual entry, map coordinates, external service failure, and zone assignment on real devices; a GPS pin is not guaranteed to fill every address label correctly.
WhatsApp/social/media	wa.me support links, Instagram links, remote Pexels/Unsplash images, and Pravatar demo imagery appear in source.	Confirm official links, contact details, consent-safe message text, and image rights/fallbacks; these are not messaging API integrations.

Critical repository cautions

- Edge Function source is split.** thk-web/supabase/functions/ contains tap-checkout; thk-mobile/supabase/functions/ contains additional functions including tap-webhook, send-reminders, send-booking-notification, reconcile-activity, and process-account-deletion. Do not deploy one folder blindly. Confirm which code is deployed to the production Supabase project, diff the duplicates, choose a canonical source, and bring the other tree into agreement.
- Migration history may be incomplete.** The repository README records missing-local migration history in the production Supabase project. Do not run a bulk supabase db push until the remote/local history is reconciled. Review each migration and apply through Triangle Healthy Kitchen's approved release procedure.
- Database migrations are duplicated.** Compare web and mobile migration trees before database changes; apply each intended migration once to the correct Supabase project.
- Current repository contains local build artifacts and configuration outside this documentation set.** Do not distribute node_modules, dist, www, .temp, .env*, google-services.json, signing files, or local IDE/AI state as handover material.
- Demo SQL scripts are destructive or mutate demo records.** Read their comments and target filters; do not run against live customers.

Useful source entry points

- Web routing and customer pages: thk-web/src/App.tsx, src/pages/, src/components/
- Staff access: thk-web/src/lib/auth.tsx

- Operations workspaces and role modules: `thk-web/src/pages/OperationsWorkspace.tsx`
- Meal signup and payments: `thk-web/src/components/SubscriptionFlow.tsx`, `src/pages/PaymentCallbackPage.tsx`
- Kitchen data transforms/views: `thk-web/supabase/migrations/`
- Mobile role routing and native integration: `thk-mobile/src/App.tsx`, `src/lib/health.ts`, `capacitor.config.ts`

> **For the Director.** This guide explains which accounts and responsibilities stay with Triangle Healthy Kitchen, how an appointed technical owner should release changes safely, and what ongoing support needs a separate written agreement. The commands below are for the developer or technical owner you appoint; the Director should not need to run them personally.

Ownership, Releases, and Maintenance

Account ownership

Triangle Healthy Kitchen should own the production Vercel project and domain, Supabase project, payment account, email sender, Firebase project, Apple Developer account, Google Play Console account, and signing credentials. If you retain a developer, invite that person as a named user with only the access required. Keep a separate secure inventory of owners, billing contacts, recovery contacts, MFA, and renewal dates; do not put passwords or recovery codes in this manual.

Web development and release

The current website lives under `thk-web/` and requires Node.js 24 per its package metadata.

```
``sh cd thk-web npm install npm run lint npm run build npm run preview ``
```

Before a production release:

1. Review the code change and database impact.
2. Confirm required environment variables in the correct Vercel environment (preview and production are separate).
3. Review and apply any database migration through the agreed release process. Back up first; check remote migration history because the local README reports historical drift.
4. Deploy a Vercel preview and run the relevant customer/role workflow checks.
5. Promote/deploy the approved commit to production using the Triangle Healthy Kitchen's Vercel workflow.
6. Verify the deployed commit, login, customer signup, menu, payment return, operations roles, and mobile-web entry point.
7. Record deployment date, commit, migration IDs, checks, owner, and rollback target.

To roll back a web-only defect, use Vercel's previous known-good deployment. A Vercel rollback does not reverse database changes; use a reviewed forward fix or a verified database restore plan.

Mobile build and release

The mobile application is under `thk-mobile/`. Android and iOS share the customer UI, but native signing, permissions, Firebase, HealthKit/Health Connect, and store policies are platform-specific.

```
``sh cd thk-mobile npm install npm run typecheck npm run lint npm run build npx cap sync ``
```

- Android emulator/debug APK: `npm run build:apk` (requires Android SDK/Gradle environment).
- Android store bundle: use Android Studio **Build -> Generate Signed Bundle / APK**, with the Triangle Healthy Kitchen's Play Console app and protected release keystore.
- iOS assets sync: `npm run build:ios`; open `ios/App/App.xcworkspace` in Xcode, set the client Team/signing profile/capabilities, archive, and upload using the Triangle Healthy Kitchen's Apple account.
- Increase the platform build/version number for each store update. Preserve and back up the same Android signing key; losing it can block updates.
- Test signed release builds on physical iPhone and Android devices. Emulator preview is not store acceptance.
- Store listing, privacy disclosures, age/content ratings, screenshots, review, and account fees are client-owned dependencies unless a separate written delivery includes them. Store publication has not been established by this handover.

Supabase/database/functions release

1. Identify the canonical migration and Edge Function tree first. The repository currently duplicates migrations and splits functions between web and mobile folders.
2. Compare local files with the deployed production function versions and deployed migration history.
3. Back up or confirm point-in-time recovery is available; rehearse a restore to a non-production environment.
4. Review policies, function grants, triggers, cron jobs, and secrets for every change.
5. Apply one reviewed migration at a time through the Triangle Healthy Kitchen's approved procedure. Do not bulk push while migration history is unresolved.
6. Deploy the intended functions and inspect function logs.
7. Confirm send-reminders-job, kitchen-choice default processing, account data cleanup schedule, and any required network extension jobs in the production project.
8. Verify exact post-release role access and audit records.

Never copy production database data into preview/local without sanitization. Do not manually update payment/subscriber tables to bypass RPC validation and audit history.

Backups and recovery

- Define a client-owned backup schedule and retention period based on the Supabase plan and the business's legal/accounting needs.
- Confirm database backup/PITR capability and storage backup coverage; database backups may not include all Storage objects or secrets.
- Store export/encryption keys separately from backups.
- Test restoration into a separate Supabase project before calling a backup usable.
- Record recovery time objective (RTO), recovery point objective (RPO), incident owner, and client notification path. These have not been contractually set in the reviewed agreements.

Routine maintenance calendar

Frequency	Owner	Work
Each menu cycle	Admin/Kitchen	Verify next week's menu release, kitchen choice, package eligibility, recipe completeness, ingredient estimates, and deadline jobs.
Daily operations	Admin/Transport/Kitchen	Review payment exceptions, missing zones, delivery assignments, failed email events, and today's production/packing totals.
Monthly	System owner	Review staff access, provider charges/renewals, Vercel/Supabase errors and usage, backups, failed jobs, and domain/email health.
Each release	Developer/system owner	Review code, migration, security policies, preview, acceptance scenarios, release notes, and rollback path.
Quarterly or after incident	Client + technical owner	Rehearse restore, rotate access where needed, review data retention, and confirm vendor/account owners.

Support and maintenance terms

No response-time SLA, included support period, maintenance price, hosting responsibility, or uptime commitment should be implied by this document. Agree those items in writing. A practical maintenance agreement should define supported hours/time zone, severity levels, response targets, included defect fixes, paid enhancement work, backups/recovery ownership, third-party account costs, security updates, and termination/handover steps.

> **For the Director and acceptance owners.** This is the sign-off record, not a list of claims that every item has already passed. Assign each item to a named owner. Mark it Pass only after the stated evidence has been checked in the intended production account or on the intended device. If an item is not complete, record the responsible person and the operational effect before accepting the handover.

Launch Acceptance and Known Limits

The Director assigns each section to named owners from Triangle Healthy Kitchen's business, Kitchen, Transport, Finance, and technical teams. Mark each item **Pass**, **Fail**, or **Not tested** and attach a safe reference (no passwords, card data, or client health records).

A. Ownership and access

- [] Client owns Vercel/domain and can deploy or roll back.
- [] Client owns Supabase and can manage users, billing, backups, and secrets.
- [] Client owns Tap, Brevo, Firebase, Apple Developer, and Google Play accounts as applicable.
- [] Staff have individual accounts and correct CEO/Admin/Kitchen/Transport roles.
- [] Role access checked in both UI and database (Kitchen cannot access customer/transport-only information; Transport cannot approve payments).
- [] Rider application approval has a confirmed supported workflow. **Current operations source only loads approved riders; pending approval control was not found.**
- [] Recovery contacts and MFA set; departing staff can be removed without sharing/resetting other people's credentials.

B. Customer signup and account

- [] English and Arabic landing/navigation, forms, errors, legal links, and account views reviewed on mobile and desktop.
- [] Email signup/login, password reset from the live domain, and session recovery tested.
- [] Phone country-code selection and OTP delivery channel tested using production provider settings; verify whether WhatsApp OTP is actually configured before promising it.
- [] Profile fields reject invalid/zero weight or height and explain optional BMI report clearly.
- [] Existing customer can sign in, renew, see their plan, and request deletion without being forced into a duplicate purchase.
- [] Account deletion request and 30-day inactive-customer cleanup are tested; published policy and actual cron schedule match.

C. Plan, menu, and kitchen

- [] Active package prices, currency, service days, meal periods, Friday eligibility, and meal limits match business-approved values.
- [] One-day and six-day trials show 3 meals + 1 snack if confirmed by the business.
- [] 1,600 kcal, 3-meal/2-different-snack package is active at the approved price (currently requested as QAR 2,499).
- [] Friday add-on is QAR 199 per monthly subscription and includes Friday meals/selections and one Friday delivery stop per service Friday.
- [] Admin menu import/upload, editable review, additions/deletions, publication, Saturday customer availability, and future week selection tested.
- [] Previous release fallback fills a missing week and does not duplicate menu-availability rows.
- [] Customers can keep preselected Kitchen choice, edit desired meals, and cannot exceed package meal periods.

- [] Weekly cutoff is Thursday 11:59 PM Qatar time; job applies Kitchen's choice only after cutoff; reminders arrive approximately 24 hours beforehand.
- [] Kitchen sees today's required dish/meal totals and relevant allergy/preparation notes; customer addresses/zones remain with Transport/Admin.
- [] Purchase quantities use kitchen-verified recipe amounts/units, include substitutions/removals correctly, and handle portion A-F scaling as intended.
- [] Packing shows client-specific dish and portion details needed after bulk purchase.
- [] Friday delivery list includes only monthly add-on customers; delivery is one stop per customer/day with all meals together.
- [] Test at least two customers with different dishes, shared dishes, different A-F portions, allergy notes, customizations, and different zones.

D. Payments

- [] Tap production keys and account settings verified in Tap and stored only in server-side configuration.
- [] Tap redirect URL is the live domain callback and Tap webhook points to the correct production Edge Function.
- [] A permitted live test produces the correct transaction, currency, package/Friday total, callback, webhook, and plan activation.
- [] Delayed callback case stays pending and is reconciled without asking the customer to repurchase.
- [] CEO/Admin manual Tap verification works only after a human confirms the exact captured charge, currency, and amount.
- [] Cash stays pending until physically collected; CEO/Admin verification activates it and creates an audit event.
- [] Failure, duplicate callback, cancellation, refund, and chargeback procedures are agreed. Refund/chargeback UI/process is not established by this documentation review.
- [] Sadad is not described as available unless separately implemented and acceptance-tested.

E. Email, mobile, and integrations

- [] Brevo API configuration, sender/domain verification, booking notifications, menu reminder delivery, and failure logs tested.
- [] Supabase cron jobs verified in production: weekly default, send-reminders, and 30-day cleanup.
- [] Canonical Supabase Edge Function tree chosen and deployed; `tap-webhook` and other mobile-tree functions are not assumed to exist just because files are in the repo.
- [] Android Firebase config belongs to client's project, is secure, and push is tested on a real device.
- [] Apple Developer team, app identifiers, HealthKit capability, privacy usage text, and signed iOS build are configured.
- [] Apple Health and Android Health Connect consent/read/sync are tested on real devices; Samsung Health sharing steps documented for users.
- [] Address autocomplete/locate/manual entry and Qatar zone mapping verified with real addresses; map marker accuracy checked.
- [] Store privacy, data safety, account deletion, nutrition/health, payment, and location disclosures match actual collection/use.

F. Release and operations acceptance

- [] Web build, role sign-in, customer signup, menu, payment, kitchen, route, Arabic, and mobile responsive checks pass.
- [] Android signed build installed/tested on physical phone and release signing key backed up.
- [] iOS signed build installed/tested on physical iPhone. Store publication is separately scheduled with client-owned account.

- [] Demo customers are labeled unpaid and excluded from sales totals.
- [] Customer, payment, kitchen, packing, and route CSV exports open and contain expected columns.
- [] Backup restore is demonstrated outside production.
- [] Operations and client-owner training completed; support owner and escalation contact named.
- [] Client accepts the agreed scope and signs the aligned commercial/license documents.

Known limitations and items requiring acceptance

1. **Payment provider:** Tap implementation is present but live transaction/callback behavior needs Triangle Healthy Kitchen's credential and webhook verification. Sadad was not found in application files reviewed for this handover.
2. **Supabase functions are split:** checkout appears under the web tree; webhook, reminders, booking email, health reconciliation, and account deletion appear under the mobile tree. Production deployment must be reconciled with this layout.
3. **Migration history:** root README warns that the linked production project has local/remote migration-history drift. Bulk database push is not a safe assumed handover step.
4. **Menu/kitchen data quality:** recipe ingredient quantities are seeded estimates and require Kitchen approval. Portion ranges A-F require real gram values. Procurement should not rely on defaults until confirmed.
5. **Rider approval:** the reviewed operations dashboard shows approved riders but no pending application approval action. Confirm a usable approval process before onboarding riders.
6. **Mobile publication:** source projects exist but the reviewed state does not establish signed, store-approved iOS/Android releases. Client store memberships and signing are prerequisites.
7. **Push notifications:** Android registration depends on Firebase configuration; the Firebase config file is deliberately excluded from this package.
8. **Arabic completeness:** translation infrastructure and many Arabic strings exist. A complete screen-by-screen Arabic review is still an acceptance task; this package does not certify every string.
9. **Operational reminder scheduling:** SQL/function code exists; actual production cron, secrets, and delivery require verification.
10. **Support/warranty:** response times, uptime, included maintenance period, and ongoing fees are not established here. Agree them separately in the signed commercial terms.

Sign-off

Area	Result	Owner	Date / evidence reference
Customer signup and plans			
Menu and Kitchen			
Payments			
Transport and riders			
Arabic and mobile			
Deployment, backups, and recovery			
Client acceptance			

> **For the Director and provider-account owners.** These services are operated through separate vendor accounts. Triangle Healthy Kitchen should own each production account and billing relationship. Assign the steps to the named account owner or technical owner; never send credentials to the developer through this manual, email, screenshots, or source code. The runbook says what each service does, what to verify, and what to do when it fails.

Outside Services Setup and Support

This runbook identifies services outside the application code, what THK uses them for, who should own them, how to configure or verify them, and what to do when they fail. Create and retain every production provider account in Triangle Healthy Kitchen's name. Keep all credentials in the provider or its approved secret store; this runbook intentionally contains no credential values.

Service inventory

Service/source	Use in THK	Key operational point
Supabase	Authentication, PostgreSQL, Storage, RLS, Edge Functions, cron jobs	System of record. Preserve migration history and verify security policies/functions before release.
Vercel	Production website hosting and deployment	Deploys the web app; keep the Supabase project and domain settings aligned.
Brevo	Supabase Auth SMTP and transactional booking/menu emails	SMTP key and API key are different credentials and are consumed through different paths.
Tap Payments	Online checkout, charge status, webhook/callback	Reconcile exact captured charges; never trust the browser return page alone.
Firebase Cloud Messaging	Android push registration	Android push is skipped unless <code>google-services.json</code> exists at the expected native path.
Apple HealthKit	Optional iPhone health-data read/sync	Requires client Apple Developer membership, HealthKit entitlement, signing, and user consent.
Android Health Connect	Optional Android health-data read/sync	Samsung Health must share data with Health Connect first.
Qatar GIS zones service	Converts a map point into Qatar ZONE_NO	Source code uses the Qatar GIS public FeatureServer query; if unavailable, zone stays unset and Transport must review.
OpenStreetMap Nominatim	Reverse-geocodes GPS coordinates into address labels	Current source calls the public reverse endpoint without an API key. Treat the result as a suggestion, provide manual entry, and check current usage terms/provider suitability before higher-volume production use.
Google Maps	Opens/parses customer/rider map links and coordinates	Current source builds links and parses coordinates; it does not show a Google Maps API key or Places autocomplete integration.
WhatsApp (wa.me)	Opens a prefilled support or rider-contact conversation	This is an outbound deep link, not evidence of WhatsApp Business API or WhatsApp OTP delivery. Check current number and privacy-safe message content.
Pexels / Unsplash	Food/marketing fallback images	Remote image availability can affect presentation. Replace with licensed, client-approved images for a production brand.
Pravatar	Community demo avatar images	Synthetic/community presentation use only; do not represent generated avatars as real named customers.
Instagram	Footer/social navigation	Confirm one correct official handle on web and mobile before launch; the source contains different web/mobile handle strings.

1. Supabase

Ownership and initial access

1. Keep the production project under Triangle Healthy Kitchen's organization, with at least two client-controlled owner/recovery contacts and MFA.
2. Invite developers as named team members with only the access needed. Do not share the owner login.
3. Keep the project URL and anon/public key in the client app configuration. Never use the service-role key in web/mobile code.
4. Store server keys only in Supabase Edge Function secrets or the protected database Vault/RPC mechanism expected by the function. The service-role key bypasses ordinary row policies and must never be sent to a browser/device.

Auth and email setup

1. Set the production Site URL and redirect allow-list to the deployed domain and supported auth/payment return paths.
2. Test signup confirmation, sign-in, password recovery, session expiry, and callback handling on the live HTTPS domain. A recovery link must return to a valid app session and route.
3. If using custom SMTP for Supabase Auth, use Brevo's **SMTP relay hostname** (`smtp-relay.brevo.com`) and the SMTP port supported by the account (the configured screen previously showed 587). Use Brevo's SMTP username and a generated SMTP key in Supabase; this password is **not** the Brevo API key and is not the account login password.
4. Confirm sender/domain authentication and that Supabase Auth confirmation/reset messages arrive independently from THK booking/menu transactional messages.
5. Phone/WhatsApp OTP delivery depends on Auth provider setup. A WhatsApp support link in the app does not configure OTP delivery.

Database, storage, functions, and scheduled jobs

1. Reconcile local and remote migration history before applying migrations. The repository records historical drift; do not use an unreviewed bulk push.
2. Review RLS and RPC permissions after each migration. Check as anonymous, customer, Kitchen, Transport, Admin, and CEO accounts.
3. Confirm the `bmi-reports` Storage bucket and policies restrict each uploaded report to the intended authorized flow. Treat reports as sensitive health data.
4. Reconcile the split Edge Function trees in the repository before deployment (see Technical Handover). Check production function source/deployed versions rather than assuming a local copy is live.
5. In Supabase logs, verify function invocations/errors and the cron jobs for menu defaults, menu reminders, and expired-customer cleanup. Confirm they target this production project and use the correct Qatar-week logic.
6. Confirm backup/PITR availability, storage coverage, and a restore drill. A database backup may not restore object files, function secrets, or external provider configuration.

If something fails

- **Auth redirect/reset failure:** check Auth Site URL/redirect allow-list, SMTP logs, and whether the recovery link is stale; generate a fresh recovery link.
- **HTTP 401/403:** verify the current user/session, role assignment, RLS policy, and RPC grants. Do not solve by exposing service-role credentials.
- **HTTP 400 on a table/RPC:** read the Supabase error body and compare the deployed schema/function with local migration history before changing data.
- **Cron does not run:** inspect `cron.job` and `job-run/network` logs, function endpoint, secrets, and `pg_net/extension` configuration.

2. Vercel

1. Keep the Vercel project under Triangle Healthy Kitchen's account or organization and connect the intended GitHub repository/production branch.
2. Set the project root to `thk-web/` if that is not already the configured Vercel root. Confirm the build command is `npm run build`, output directory is `dist`, and SPA routes rewrite to `index.html`.
3. Configure `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` in Preview and Production. Set `VITE_OPERATIONS_WEB_URL` in the mobile build only if the default production URL should be overridden.
4. Use a Preview deployment to verify each release before promoting to Production. Confirm `/login`, `/account`, `/payment/callback`, `/dashboard`, and role-specific routes load directly.
5. After a production deploy, record the Git commit and deployment URL; exercise sign-in, meal selection, and payment-return routing.
6. Vercel Analytics can be blocked by browser privacy extensions (`ERR_BLOCKED_BY_CLIENT`). Distinguish that client-side block from application/API failures; enable analytics for the Vercel project only if the owner wants it.
7. Roll back a web-only regression to a prior Vercel deployment. Database changes need a separate reviewed recovery/fix; a frontend rollback does not reverse SQL.

3. Brevo

THK uses Brevo through **two distinct channels**. Configure and test both:

Supabase Auth transactional SMTP

- Set a verified sender/domain in Brevo and finish the DNS authentication steps Brevo requests.
- In Supabase Auth email/SMTP settings, enter Brevo's SMTP relay, the account's SMTP username, and a generated SMTP key.
- Use the SMTP key, not the Brevo dashboard password and not the API key.
- Test a new-user confirmation and a password reset on the production domain. Check spam placement, From name/address, reply handling, and rate limits.

THK Edge Function emails

- The booking-notification function source reads `BREVO_API_KEY` and `SENDER_EMAIL` from Edge Function environment secrets.
- The menu-reminder function source calls the protected `get_brevo_config()` database helper, which reads a key/sender from the project's Vault configuration.
- Because the two source copies use different configuration paths, first reconcile the canonical production function source. Put the API key only in the exact server-side secret/Vault location used by that deployed function. Do not paste it into `VITE_*`, frontend code, GitHub, or this manual.
- Confirm `send-booking-notification` sends the expected customer/operations booking messages and writes success/failure logs.
- Confirm `send-reminders` runs from the scheduled job, selects only customers who still need to choose meals, sends before the Thursday Qatar-time cutoff, and records the `(customer, service week)` reminder so a repeated invocation does not spam them.
- Check Brevo transactional email logs and the THK Notifications view for accepted, bounced, blocked, or failed outcomes. A 2xx provider response means accepted for processing, not guaranteed inbox placement.
- Rotate a compromised API key in Brevo and in the correct Supabase secret store, redeploy/restart dependent functions if needed, then send a controlled test.

4. Tap Payments

1. The client owns the Tap merchant account and activates live processing with Tap. Confirm the environment (test/live), enabled payment methods, currency, settlement, and account status directly in Tap.

2. Store the Tap secret only through the protected Supabase configuration path used by `get_tap_config()`; confirm the deployed checkout function can retrieve it. Do not place the secret in Vercel `VITE_*`, app source, or a mobile bundle.
3. Set the redirect URL to `https://trianglehealthykitchen.vercel.app/payment/callback` (or the approved active domain).
4. Set the webhook/post URL to the correct production Supabase `tap-webhook` Edge Function. Verify the URL and signing/HMAC behavior against the deployed function. The current checked-in webhook copy is under `thk-mobile/supabase/functions/`; reconcile it with the deployed function before editing/deploying.
5. Make one authorized end-to-end payment using an approved test/low-value method. Confirm the transaction row, Tap dashboard charge, webhook/callback logs, exact amount/currency, and active customer plan.
6. Simulate delayed callback behavior without creating a second charge. CEO/Admin manually reviews only after confirming the exact charge is captured in Tap.
7. Agree who handles refunds, disputes, chargebacks, settlement reconciliation, and customer receipts. Those procedures are not fully represented by the current app screens.
8. Sadad is not in the reviewed app code. Do not add Sadad credentials or advertise it as integrated until a separate implementation is built and tested.

5. Firebase Cloud Messaging (Android)

1. Create/choose the Triangle Healthy Kitchen's Firebase project and register the Android app with package/application ID `com.trianglehealthykitchen.app`.
2. Download the Android client configuration file from Firebase and put it at `thk-mobile/android/app/google-services.json`. Keep it out of public Git and out of this documentation archive.
3. Confirm Gradle's Google Services/Firebase Messaging setup and the application ID match Firebase. Rebuild and sync the Capacitor project after adding configuration.
4. Sign in on a physical Android device, grant notification permission, confirm FCM token registration, then send a controlled test message through the intended server notification path.
5. If there is no Firebase configuration, current app code intentionally skips Android push registration. Email reminders continue as a separate channel; do not claim Android push is live until tested.
6. Store Firebase Admin/service credentials, if later needed by server push, only in a protected server secret store. The Android client JSON is not a server admin key, but it still belongs to the client's project and should not be published casually.

6. Apple Health and Android Health Connect

iPhone / Apple Health

- The app asks for customer consent to read steps, distance, calories, and weight. It does not sync until the customer initiates it.
- The iOS project has a HealthKit entitlement and usage description. Triangle Healthy Kitchen still needs an active Apple Developer membership, the matching App ID capability, provisioning/signing, and a signed device build.
- As of this handover, the Apple Developer subscription had not yet been paid. Therefore iOS HealthKit store/distribution readiness is blocked until that account is active and signing is configured. A simulator/source check is not a production-device validation.
- Test grant, deny, revoke, empty data, and successful sync on an iPhone. Confirm the privacy policy accurately describes health data.

Android / Health Connect

- The app queries Health Connect through the Capacitor health plugin after the user grants access.
- On Samsung phones, the user must enable Samsung Health sharing into Health Connect first. Test on a physical Samsung/Android device as well as a generic Android device.
- Test unavailable Health Connect, denied permissions, no samples, and successful daily steps/distance synchronization.

Health integrations are optional wellness features; do not present their output as diagnosis or medical advice.

7. Location and maps

- The app can use device geolocation, parse Google Maps URLs/coordinates, and create outgoing Google Maps links. The application files reviewed for this handover do not use a Google Maps API key or provide Places autocomplete.
- The `Locate` flow calls OpenStreetMap Nominatim reverse geocoding to suggest address labels, and Qatar's GIS FeatureServer to resolve a `ZONE_NO` from the GPS point. These are external network services; they can fail, time out, or return incomplete address data.
- If either lookup fails, the customer must be able to enter/review the delivery address manually. Treat zone as unconfirmed if the lookup is blank; Transport should check it before assigning a rider.
- Verify coordinates inside Qatar, edge-of-zone addresses, a manually entered home/office/gym address, denied location permission, and network failure. Ensure the location pin and human-readable address refer to the same place.
- Before increasing traffic, check current Nominatim and Qatar GIS terms/availability and decide whether the business needs a supported geocoding service or server-side proxy. Do not assume a public endpoint has a commercial uptime commitment.

8. WhatsApp, social links, and remote images

- WhatsApp buttons open `wa.me` chats; this is not a WhatsApp Business API integration. Confirm the business number, country-code formatting, prefilled text, and rider/client support routing.
- Do not place passwords, payment-card data, health measurements, BMI reports, or detailed allergy information in WhatsApp URLs/messages. The current client-support link can include the customer's name in the prefilled text; review consent and minimize data in these message templates.
- Confirm website and mobile Instagram links point to the same official account. The web and mobile currently contain different handle strings between the two clients.
- Pexels/Unsplash supply remote image URLs; Pravatar supplies generated community avatars. Ensure the owner has appropriate usage rights/approval and replace demo-looking assets before client sign-off. Remote images require network access; test their fallback/loading behavior.

Provider incident checklist

For any external service failure, record: provider, affected environment, time in Qatar, affected workflow, safe transaction/request reference, error code, recent deployment or key rotation, and whether a fallback worked. Do not attach tokens, keys, password-reset links, full payment details, medical reports, or screenshots with customer PII. Assign one owner to contact the vendor and another to keep customer operations moving using the documented fallback.